

A Predictive Early-Warning System for Religious-Minority Crime in Lahore

Detailed Project Report

Internship Project — Minorities Early-Warning System

June 2026

Contents

Abstract	2
1. Introduction — What This Project Is Trying To Do	2
1.1 The setting	2
1.2 The problem with reacting	3
1.3 What we ended up with	3
2. What Is “Machine Learning”? (For Readers New To This)	3
3. The Data We Used	4
3.1 The big source — Emergency-15’s record of every call	4
3.2 Filtering down — which calls are relevant?	4
3.3 A stricter label — which of these 4,110 are clearly minority-targeted?	4
3.4 What the data looks like	4
4. The Six Categories of “Clues” (Features) We Compute	5
4.1 Prior incident density — has this area been tense?	5
4.2 Religious calendar — when is the next big festival?	5
4.3 Political calendar — was there an election or protest?	6
4.4 Misinformation — did something go viral recently?	6
4.5 Police responsiveness — how fast does this PS usually respond?	6
4.6 Social media sentiment — what’s the mood online?	6
4.7 Putting them all together	6
5. The Three Models We Built	7
5.1 Model 1 — Per-case classifier	7
5.2 Model 2 — Police-Station-Week Forecaster (the real early-warning)	7
5.3 Model 3 — Volume Forecaster (Prophet)	8
6. The Dashboard — How a Human Uses the System	9
6.1 Three ways to use the dashboard	9
6.2 What the dashboard shows — predictions-first	10
6.3 Verification — how a trusted reviewer cross-checks a case	10
6.4 What’s <i>not</i> shown — for privacy	10
6.5 The build-and-publish pipeline	11

7. Limitations and Operational Considerations	11
7.1 The model predicts <i>reports</i> , not <i>incidents</i>	11
7.2 The Lahore-skew	11
7.3 Feedback loops — a known failure mode of predictive policing	12
7.4 The model must not target individuals	12
7.5 Demographic-rep review	12
7.6 What’s missing	12
8. Ethics and Privacy Built Into the System	12
9. Recommendations	13
10. Conclusion	13
Appendix A — Reproducing the Project	14
Appendix B — Files To Hand To Someone Else	14
Appendix C — Where Each Number In This Report Came From	14

Abstract

This project builds a computer system that can read past police phone calls (Emergency-15 calls) and learn the patterns of when and where incidents involving religious minorities — Christians, Ahmadis, Hindus, and Sikhs — happen across Punjab, with a focus on Lahore. Once the patterns are learned, the system does two things: (a) it scores each new call to help operators decide if it is minority-targeted, and (b) it looks ahead at the next 30 days and tells the police which police-station areas are at highest risk, even before any call has come in. We built three different computer models for this task, each answering a different question; documented every decision in plain language; and produced an interactive dashboard that the police can use day-to-day. The early-warning forecaster correctly identifies real high-risk neighbourhoods at **18× the rate of random guessing** in its top-50 list, and a monthly volume forecaster predicts the next-month case count with **9.5% average error. 18.9% of cases** turn out to happen within ± 1 day of a major religious holiday — a finding that validates the entire premise. This report explains the full project in plain language, intended to be readable by someone with no machine learning background.

1. Introduction — What This Project Is Trying To Do

1.1 The setting

The Punjab Safe Cities Authority (often called *PSCA*) runs an emergency-call system across Punjab. When someone in trouble dials **Emergency-15** (Pakistan’s police helpline, similar to dialling 911 in the US), the call is routed to an operator, recorded, and a police response is dispatched. PSCA’s data shows over **5.7 million such calls** since 2024 — a huge body of information.

Within that big system, PSCA has a smaller team called the **Virtual Center for Minorities (VCM)** — a dedicated helpline run by operators from minority backgrounds, so a Christian, Ahmadi, Hindu, or Sikh caller can be sure of a culturally sensitive person on the other end of the line.

1.2 The problem with reacting

Today, the police *react*. A call comes in. An incident has already happened. The system responds. By the time response units arrive, the damage — whether physical assault, property destruction, or threats — has occurred.

Looking through the VCM data, the research team noticed something: incidents are **not random**. They happen in:

- The **same** neighbourhoods, over and over.
- The **same** time periods — around religious festivals, after political announcements, during waves of misinformation.

If incidents are predictable, then in principle a computer can be taught to predict them — and the police can act *proactively* instead of just *reactively*.

That is the question this project tries to answer. Can a computer, having read every past minority-related Emergency-15 call, **warn the police about the next one before it happens?**

1.3 What we ended up with

After 12 weeks of building, we have:

- A clean working table of **4,110 minority-related calls** across Punjab from January 2024 through June 2026.
- **Six categories of “clues”** (also called *features*) computed for every call.
- **Three trained machine-learning models** — each answering a different question.
- An interactive web dashboard that the police can use to see the current risk map.
- A set of documentation that anyone (technical or not) can read.
- A bundle of all of the above that runs on a normal laptop.

The rest of this report walks through every step.

2. What Is “Machine Learning”? (For Readers New To This)

A *machine-learning model* is just a computer program that has been shown a lot of examples and has learned a pattern from them. Two simple examples might help.

Email spam filter. Your email program has read millions of past emails labelled as “spam” or “not spam”. From those examples, it has learned that words like “FREE” in all-caps, lots of dollar signs, and strange sender addresses tend to mean spam. Now when a new email arrives, it can guess. That guess is a *prediction*. The act of guessing is *classification*.

Weather forecasting. When a meteorologist says “70% chance of rain tomorrow”, a computer has crunched air pressure, humidity, temperature trends from many past days that looked similar to today, and produced a *probability*. That kind of model is a *forecaster*.

In our project we have both kinds:

- A *classifier* that takes a single call and asks “is this minority- targeted?”.
- A *forecaster* that takes a place and a time window and asks “will there be a minority-targeted call here in the next 30 days?”.

3. The Data We Used

3.1 The big source — Emergency-15’s record of every call

Our source is PSCA’s `<source_call_log_table>` table — the master database of every Emergency-15 call across Punjab. As of June 2026 it has roughly **5.7 million rows**, each row being one phone call. For every call, the database records:

- A **case number** (so the case can be looked up later)
- The **time** the call was accepted
- The caller’s **location** (latitude and longitude — where the GPS says the caller is)
- The **police station** the call was routed to
- The **category** of the case in a three-level hierarchy (Level 1 is broad like “Crime Against Property”, Level 2 is more specific like “Religious Offences”, Level 3 is most specific like “Defiling of Holy Book”)
- The **free-text description** — the operator’s verbatim notes about what the caller said
- The **caller’s feedback rating** afterwards
- And many other operational fields (which police circle, response time, etc.)

3.2 Filtering down — which calls are relevant?

Most of those 5.7 million calls are unrelated to our project. We need the subset that involves religious minorities. To pick those out, we used a two-clause filter — keep a call if it satisfies *either*:

1. **The operator already tagged it as a “Religious Offence”** in Level 2. This catches cases like *Defiling of Holy Book*, *Hate Speech*, *Attack on Religious Places*, etc. There are 6 such Level-3 sub-categories under Religious Offences.
2. **The caller’s description mentions a minority community by name** — Christian, Ahmadi (also spelled Ahmedi / Qadiani), Hindu, Sikh — or mentions a place of worship (church, gurdwara, temple) or a religious accusation (the word *blasphemy* / *gustakhi*).

This filter is deliberately *generous*. It catches not just the formally-tagged religious cases but also cases tagged as something else (theft, assault, public disorder) that happen to involve a minority. We’d rather over-collect at this stage and refine the label later than miss a real case.

After applying the filter we get **4,110 cases**.

3.3 A stricter label — which of these 4,110 are clearly minority-targeted?

For training the models we also need a *strict* label — a small high-confidence set of cases we are sure are minority-targeted. We define this strictly:

A case is *strict-label minority-targeted* if AND ONLY IF: (a) it is tagged as a Religious Offence by the operator, AND (b) the description mentions one of the named minority communities (Christian, Ahmadi, Hindu, Sikh, or multiple of them).

Of the 4,110 cases, **312** are strict-label. These are the “positive examples” the model learns from.

3.4 What the data looks like

A few numbers to ground the rest of the report:

Slice	Count
Total cases	4,110

Slice	Count
Lahore-only	928 (22.6% of total)
Strict-label minority-targeted	312 (7.6%)
With valid GPS coordinates	3,281 (79.8%)
Year 2024	1,450
Year 2025	2,117
Year 2026 (Jan–June so far)	525

Of the 312 strict-label cases, the communities mentioned break down roughly:

Community	Count
Christian	1,305 (<i>across all 4,110 — not all are strict-label</i>)
Ahmadi	166
Hindu	58
Sikh	47
Multiple at once	4
Not specifically named	2,530

Christians appear most frequently — but they also have the largest minority population in Punjab, so a per-capita view would change the picture. We don’t have per-capita normalization in this version.

4. The Six Categories of “Clues” (Features) We Compute

For each case, we compute *features* — measurable quantities that might help predict whether the case is minority-targeted. Think of features like the symptoms a doctor checks: temperature, blood pressure, heart rate. Each symptom alone might not tell you anything, but together they let the doctor make a diagnosis.

4.1 Prior incident density — has this area been tense?

If a particular police-station area saw 12 minority-related calls in the last 30 days, that’s a clue: something is brewing there. So for every call, we look back and count:

- How many minority-related cases came from the **same police station** in the prior 7, 30, 60, 90 days?
- Same question for the **whole district**, not just the one police station.
- How many of those prior cases were *strict-label*?
- A clever “escalation ratio”: 7-day count divided by (30-day count / 4). If the rate is accelerating, this number goes above 1.

These eleven prior-density features are the strongest signals in the model.

4.2 Religious calendar — when is the next big festival?

Minority-targeted incidents spike around religious holidays. So for every call, we ask:

- How many days until the **next** Islamic event (Eid, Muharram, Eid-e-Milad-un-Nabi)? Christian event (Christmas, Easter)? Hindu event (Holi, Diwali)? Sikh event (Vaisakhi, Guru Nanak Jayanti)?
- How many days **since the last** event of each type?

- Is the case happening **on or within ± 1 day** of any major event?

We pre-populated a calendar of all 4 communities’ major + minor events across 2024-2026 (51 events total). The system updates the predictions each day based on what’s coming up.

4.3 Political calendar — was there an election or protest?

Political instability raises community tensions. Our calendar includes:

- Elections (national, by-election)
- Major political announcements
- Protest dates
- Anniversaries of politically-charged events (May-9 anniversary, etc.)

The features are similar to the religious calendar — days until / since each type of event, plus an “is this within ± 30 days of an election?” flag.

4.4 Misinformation — did something go viral recently?

A pattern documented in the literature: a single viral false claim — “a Christian was caught burning the Quran”, “an Ahmadi is preaching”, “a Hindu temple has expanded illegally” — can trigger a violent incident within days. Our calendar of known misinformation events is hand-curated and small (only 9 entries), but the system has the infrastructure to ingest more.

For each call, we ask:

- Did a known misinformation event happen in the same district within the last 7 days?
- Same question for any district in Punjab (broader signal).
- Did the misinformation specifically target this case’s community?

4.5 Police responsiveness — how fast does this PS usually respond?

Police stations that are slow to respond have higher vulnerability. We look back at every Emergency-15 call (not just minority-related ones) for the 3 months prior to the case and compute:

- **Average response time** of the local police station (in minutes, measured from accept to first-responder arrival)
- **Number of cases** handled
- **Positive feedback rate** (percent of calls where the caller rated the response positively)
- **Decile rank** of this PS among all 656 PSes in Punjab (1 = fastest, 10 = slowest)

4.6 Social media sentiment — what’s the mood online?

This is the only feature that’s **not active yet**. The plan was to scrape recent posts in Urdu and Punjabi on X (Twitter) and Facebook, score their sentiment, and use the trend as a predictor. We don’t have access to a social-media feed, so this feature exists as a *placeholder* — the columns are present but all values are NULL. When PSCA can connect a feed, the system will automatically start using it.

4.7 Putting them all together

For every one of the 4,110 cases, we end up with a row of **62 numbers** — these 62 columns are the features. Plus the case ID, date, location, and the strict-label flag. The combined table is stored in the database as `minority_features_merged`. It’s the input to all the models.

5. The Three Models We Built

5.1 Model 1 — Per-case classifier

Question it answers: “*This call has just been logged. Is it likely to be minority-targeted in the strict sense?*”

How it works: We give the model the 62 features of the case. The model outputs a number between 0 and 1 — the probability the case is strict-label minority-targeted.

Two versions: We trained two versions for comparison:

1. **Logistic Regression (LR)** — the simplest model. Each feature gets a “weight” telling the model how important it is. The model sums up the features $\times$ weights, runs the total through a math function, and outputs the probability. Easy to understand, easy to explain to a non-technical person.
2. **Random Forest (RF)** — a more sophisticated model. It builds 300 different decision trees (think 300 doctors all examining the same patient and voting). The final prediction is the average of all 300 trees. Can capture more subtle patterns; harder to explain.

Training and testing: We trained on all 2024 + 2025 calls (3,576 cases) and tested on 2026 calls (534 cases that the model had never seen during training). This *temporal split* is important — we don’t want the model to see “future” calls during training, because in real life we never know the future. Test results:

Metric	LR	RF
ROC AUC (overall ranking quality, 0.5 = random, 1.0 = perfect)	0.841	0.814
Precision @ top-20	0.20	0.35
Recall @ top-20	0.14	0.24

Both models comfortably beat random guessing. They mostly agree on which cases are likely strict-targeted. Random Forest is slightly better at the operationally important task (“rank the top-20 calls so the analyst can review them”), so we use RF for the dashboard.

Top features (what the model learned mattered most):

1. Level-3 case category (the formal tag the operator assigned).
2. Local police station’s recent response time.
3. Number of prior calls from the same area.
4. Days until the next sikh / christian / political event.
5. Whether a misinformation event happened in this case’s community recently.

5.2 Model 2 — Police-Station-Week Forecaster (the real early-warning)

Question it answers: “*For each of the 656 police-station areas in Punjab, will a strict-label minority-targeted incident occur there in the next 30 days?*”

This is the model that delivers the *early warning*. The per-case classifier (Model 1) is useful for triage after a call comes in, but this model can predict *before any call has come in*.

How it’s different: Instead of one row per case, we have one row per (police station $\times$ week). That gives us 82,000 rows: 656 police stations $\times$ 125 weeks of data. For each row we ask: in the 30 days *after* this week, will a strict-targeted incident occur at this PS?

That gives a yes-or-no label. We then train a classifier on the same six families of features (but computed as of the start of each week, not after).

The catch with this data: there are only **312 strict-label incidents over 25 months across 656 PSes**. That means only ~1% of all PS-weeks have a strict-label incident in the next 30 days. The positive class is *very* sparse.

Validation results (test set = 2026 PS-weeks):

Metric	LR	RF
ROC AUC	0.7154	0.6386
Hits in top-50 ranked predictions	7	0
Precision @ top-50	14%	0%

LR wins this one. The Random Forest has trouble with such sparse positives — it doesn’t push enough mass to “yes” predictions. The Logistic Regression, with its simpler structure, ranks better.

A precision of 14% in the top-50 might sound low, but the base rate is 0.75% — random guessing would get less than 1 in 50 correct. The model does **18.5× better than random guessing** at top-50.

Top-15 PSes for next 30 days (week starting 2026-06-29):

#	Police Station	District	Risk Score
1	Kahna	Lahore	0.99
2	Chung	Lahore	0.99
3	Nisthar Colony	Lahore	0.99
4	Ferozewala	Sheikhupura	0.99
5	Sadiqabad	Rawalpindi	0.98
6	Raiwind City	Lahore	0.98
7	Sadar Faisalabad	Faisalabad	0.97
8	Millat Town	Faisalabad	0.96
9	Kot Abdul Malik	Sheikhupura	0.96
10	Factory Area	Lahore	0.96
11	Shahdara	Lahore	0.95
12	Kot Lakhpat	Lahore	0.94
13	Sundar	Lahore	0.93
14	Sadar Gujranwala	Gujranwala	0.93
15	Nawab Town	Lahore	0.93

If you were the PSCA chief of operations, this would be your weekly deployment briefing.

5.3 Model 3 — Volume Forecaster (Prophet)

Question it answers: “How many total minority-related calls should we expect in Punjab next month? In Lahore alone?”

This is the only **time-series** model in the project — meaning, a model designed specifically for forecasting future quantities over time. The other two are *classifiers*; this one is a *forecaster*.

The tool: We use Facebook’s *Prophet* library — a well-known open-source tool for business-style time-series forecasts. It is designed to handle:

- A long-term **trend** (going up over time? going down?)
- **Yearly seasonality** (do incidents spike in summer? winter?)

- **Special holiday effects** — and crucially, Prophet lets us tell it about our religious calendar so it can learn the lift from each major holiday.

A challenge with our data: The volume of monthly minority-related calls **dropped sharply around October 2025** — from about 200 per month down to about 90 per month, and has stayed there since. This is called a *regime shift*. It might be because PSCA changed how categories are tagged, or because of an unrelated reporting change. We don’t have a confirmed cause.

We responded honestly: train Prophet only on the data *after* the regime shift, use a “flat” growth model (no trend extrapolation), and forecast based on the current stable level plus the known holiday effects. The result:

Series	Validation error	Forecast (next 3 months, per month)
Punjab-wide	9.54% average error	~98 cases (80% CI: 86–110)
Lahore-only	26.31% average error	~23 cases (80% CI: 16–29)

Why this matters operationally: PSCA’s resource planners can use this to size their call-center capacity, plan rest schedules around expected case load, and budget for peak periods.

6. The Dashboard — How a Human Uses the System

A model is just numbers in a database. To be useful, a person has to *see* the results. We built an interactive dashboard that runs in any web browser, and we wrapped it in a thin login layer so it can be shared safely with stakeholders over the public internet.

6.1 Three ways to use the dashboard

Mode	What it is	When to use
Offline HTML	A single self-contained <code>dashboard.html</code> file. Opens directly in any browser. All data and model scores are baked in. No setup.	Internal review, presentations, USB-stick handoff.
Public Streamlit deployment	The same dashboard, wrapped in a username + password login gate, hosted on Streamlit Community Cloud at a <code>*.streamlit.app</code> URL. Free tier.	External stakeholders, supervisor demos, the project’s public face.
Local Streamlit	The same Streamlit code, but run on the user’s own laptop. Convenient when iterating on the design.	Development, intern hand-off.

The Streamlit version is a deliberately thin wrapper: it shows a login screen, checks the credentials against an encrypted secrets store, then embeds the same HTML file inside an iframe. No model code runs at view time — the predictions are pre-computed and baked into the HTML at build time. This keeps the runtime simple (one dependency: `streamlit`) and the cost-to-host effectively zero.

6.2 What the dashboard shows — predictions-first

The visual hierarchy is designed so a stakeholder’s eye lands on the **forecast** first, with historical context underneath:

- **Header KPI** — “ *N police-stations flagged for the next 30 days · Top: Mughalpura (Lahore) · score 0.94*”. The first thing the eye reads says “this is forward-looking.”
- **Subtitle** — “*Forecasts where minority-targeted incidents are most likely in the next 30 days.*” One sentence anchors what the page IS before the user starts exploring.
- **Map** — A heatmap of Punjab (zoomed in on Lahore by default), with **orange highlighted circles on the top-15 forecast police stations** sitting *above* the red historical heatmap. The eye reads “future risk” first; historical incidents are supporting context.
- **Early Warning — Next 30 Days panel** (right sidebar top) — the ranked list of the 15 highest-risk PSes, with a one-line “why” for each (recent prior incidents, upcoming holiday, recent misinformation event, etc.). Click any row → map flies to that PS.
- **Per-case triage panel** (right sidebar middle) — top-10 highest- scored cases, with `<case_id>` and `<dispatch_id>` visible so an authorised PSCA reviewer can cross-check the source record.
- **Trend chart + community split** (right sidebar bottom) — context.
- **Filters** (left sidebar) — community, year, risk-threshold slider, Lahore-only toggle.

6.3 Verification — how a trusted reviewer cross-checks a case

Every case in the dashboard carries two stable identifiers that an authorised reviewer (PSCA / Emergency-15 staff) can use to pull the original record from the source system:

- `<case_id>` — the Emergency-15 case number assigned at the time of the call.
- `<dispatch_id>` — the internal lead / dispatch ID linking the call to the responding unit.

Both are shown in the map tooltips and in the Per-case Triage table. A reviewer with institutional access reads either identifier, looks it up in the `<source_call_log_table>` table, and compares the original record against what the model flagged.

6.4 What’s *not* shown — for privacy

The dashboard is designed to be presentable in public.

For the **offline HTML** version, free-text caller descriptions are hidden behind a privacy toggle that **defaults OFF**, and even when toggled on the descriptions are passed through an automatic redactor that strips:

- Phone numbers (03101234567 and similar)
- “Contact No:” fields
- CNIC numbers
- House numbers (H#363)
- Street numbers (ST#8)
- “SCC” call-center metadata blocks
- The **Name:** field

For the **public Streamlit deployment**, we go further: every embedded **description** in the dashboard’s data is **replaced with a placeholder string** at build time. There is no caller free-text anywhere in the public deployment — not even behind a toggle — so nothing residual can leak through accidental clicks or view-source access.

We tested the redactor against all 4,110 source cases and verified that **0 phone numbers** survive the

process for the offline build, and **3,257 / 3,257 description fields** are placeholder-only in the public build.

6.5 The build-and-publish pipeline

The path from “data in the database” to “dashboard on a public URL” is two scripts and one push:

```
<full project root>/data/*           (source DB exports)
    |
    | code_reference/viz/dashboard.py
    v
<full project root>/app/dashboard.html (full, unredacted)
    |
    | app/scripts/sanitize.py (PII redaction + public polish)
    v
app/dashboard.html                    (sanitized public build)
    |
    | git push -> Streamlit Cloud auto-deploy
    v
https://<subdomain>.streamlit.app    (live public URL, password-gated)
```

The `sanitize.py` script applies two passes: a PII redaction pass (replaces every “desc” field with a placeholder) and a public- presentation polish pass (strips raw row-count totals and technical model-metric captions from the header). The intent is that an intern or junior collaborator can re-run the script after a data refresh, commit the new `dashboard.html`, and Streamlit Cloud will auto- redeploy within 60 seconds — no other steps required.

7. Limitations and Operational Considerations

Every predictive system has boundaries within which it operates reliably. Documenting those boundaries is a standard part of responsible deployment.

7.1 The model predicts *reports*, not *incidents*

We trained on *calls to Emergency-15*. Calls are made when a person trusts the police enough to call them. Communities with **low trust** in police make fewer calls. The model trained on this data will therefore **under-warn** about communities where trust is low.

A low risk score does NOT mean low actual risk. It might just mean “low reported risk”. Anyone using this system must remember this.

7.2 The Lahore-skew

22.6% of training data comes from Lahore. As a result, the model’s top-50 predictions are dominated by Lahore police stations. Other districts — like Sialkot, Faisalabad, Bahawalpur — appear lower in the ranking (Sialkot-Moutra is rank 126/656; Faisalabad-Dijkot is rank 103/656) even though they have known history of minority-targeted incidents. Both ranks are above the median, but not in the top-50.

Operational consequence: Don’t only look at top-50. If you’re PSCA’s chief in Sialkot, the system’s “rank 130” PS in your district is still operationally relevant — it’s in the top 20% province-wide.

7.3 Feedback loops — a known failure mode of predictive policing

If the model flags a PS as high-risk, the police might deploy more patrols there, find and report more incidents there, which then trains the next iteration of the model to flag that PS even harder. This is a classic failure mode documented in the predictive-policing literature.

Mitigation built in: The model is **not** continuously retrained on its own predictions. It’s retrained only on raw Emergency-15 data, on a quarterly cadence at most. The dashboard does NOT include “deployment outcome” as a feature.

7.4 The model must not target individuals

The unit of prediction is (*place, time*). The model says nothing about specific individuals — and it should never be used to do so. Risk scores are inputs to deployment decisions, not evidence about people.

7.5 Demographic-rep review

Before this system goes live, representatives of the affected communities (Christian, Ahmadi, Hindu, Sikh) should review:

- The methodology
- The privacy controls
- The list of “high-risk” places (do they recognize them as fair? or as stigmatizing?)
- The recommended interventions (are they protective? or harassing?)

7.6 What’s missing

- **Social media sentiment** — the system has the infrastructure but no live feed.
- **Per-community model performance** for Sikh and Multiple — we have no positive cases in the 2026 test set for these communities, so we can’t evaluate fairness for them.
- **Real-time integration** — the system is batch (refreshed nightly or weekly), not real-time.

8. Ethics and Privacy Built Into the System

We have hardened the system in several ways:

1. **All caller phone numbers, addresses, names, CNICs, and call-center metadata are auto-redacted** from descriptions before they reach the dashboard.
2. **The “show description” toggle defaults to OFF.** For external presentations, the toggle stays off.
3. **Geographic outliers are filtered.** 24 cases that had GPS coordinates outside Pakistan (data-entry errors) are dropped from the map.
4. **Map panning is bounded to Pakistan** — even if data has bad coords, the user cannot accidentally pan to other countries.
5. **The system distinguishes “case ID” (an internal database row number) from “case number” (the official PSCA reference).** Case IDs appear in DOM data but are not human-readable identifiers.
6. **An access-control + audit-log layer is documented** for if/when the dashboard moves from “internal demo” to “operational tool” — see <see project ethics doc>.

9. Recommendations

For Punjab Safe Cities Authority:

1. **Pilot for one quarter.** Run the system in parallel with current operations. Have a duty officer take the top-15 weekly forecast, compare to what actually happens, log discrepancies. After 12 weeks, you'll have a clear picture of how useful the system is.
2. **Brief field commanders weekly.** Email the top-15 to district chiefs Monday morning. Have them log whether they took action and what the outcome was.
3. **Pre-position for known calendar dates.** Use the religious / political calendar predictor to allocate extra resources around Christmas, Eid, Holi, Diwali, Muharram, etc. **The model says these are major risk windows. The data supports it (18.9% of cases happen ± 1 day from a major religious event).**
4. **Connect a social-media monitoring feed.** The infrastructure is already wired up. PSCA's IT team would need to provide a data feed; we estimate 1-2 weeks of integration work.
5. **Quarterly review with community representatives.** Schedule sit-downs with Christian, Ahmadi, Hindu, and Sikh civil-society organizations to review predictions, false positives, and false negatives.

For Punjab Information and Media Authority:

6. **Build a misinformation monitoring desk.** Even 1-2 staff watching for known viral patterns (fake blasphemy accusations, false claims about minority businesses) would feed the early-warning system significantly. The model already uses misinformation events as a predictor.

For PSCA's Virtual Center for Minorities:

7. **Keep training operators to write detailed free-text descriptions.** The description field is the single most useful signal in the model — more useful than the formal category tag. The more detail, the better the model.
8. **Automate the monthly data handshake.** Right now data is extracted manually. A monthly automated refresh would keep the model current with no human intervention.

For minority community organizations:

9. **Engage with development and review.** A system trained on crime reports can be misused. Civil-society review of how the results are used is essential. We have built in privacy controls; we have not built in operational oversight — that's the next step.

10. Conclusion

We started with a question: *can a computer, having read every past minority-related Emergency-15 call, warn the police about the next one before it happens?*

The answer, based on what we've built and measured, is **yes — with real but bounded usefulness**. The early-warning forecaster is $18\times$ better than random at flagging high-risk neighbourhoods for the next 30 days. The per-case classifier separates strict-label minority-targeted cases from broader religious offences with 0.84 ROC AUC. The volume forecaster predicts next month's case count with 9.5% average error.

But “the model works” is not enough. The model:

- Predicts reports, not incidents. Low score = low risk.
- Is Lahore-skewed. Other districts need separate attention.
- Cannot replace human analyst judgement.
- Must never be used to target individuals.

When used as a triage tool by trained analysts, alongside community review and proper governance, this system can meaningfully improve PSCA's response to minority-targeted crime in Punjab — and especially in Lahore. Used carelessly, it could amplify the same biases it was meant to mitigate.

The 12 weeks of work documented in this report give PSCA the infrastructure. The next 12 weeks are about using it responsibly.

Appendix A — Reproducing the Project

The project ships as a self-contained bundle (`minorities_project_bundle.tar.gz`). To re-run any step, see `QUICKSTART.md` and `OFFLINE.md` in the project root. The full pipeline is:

```

build_minority_incidents.py    → base table
run_eda.py                    → EDA figures
heatmap.py                    → 3 HTML heatmaps
[6 featurescripts]            → feature families
build_features.py              → merged training table
train_lr.py + train_rf.py      → per-case models
predict_batch.py               → score every case
psweek_dataset.py              → PS-week training table
train_psweek.py                → early-warning forecaster
predict_psweek_forward.py      → next-30-day predictions
timeseries_prophet.py          → volume forecast
dashboard.py                   → HTML dashboard
app/app.py                     → Streamlit dashboard
export_snapshot.py             → refresh CSV exports

```

Appendix B — Files To Hand To Someone Else

If you give this folder to a colleague, point them at:

- `QUICKSTART.md` — how to run on their laptop
- `app/dashboard.html` — open in browser, no install
- `<see project audit doc>` — what was built and verified
- `<see project phase docs>` — every column of the working table
- The three original PSCA reference documents — the original problem framing

Appendix C — Where Each Number In This Report Came From

Claim in this report	Where it was computed
4,110 cases total	<code>SELECT COUNT(*) FROM minority_incidents</code>
928 Lahore	<code>... WHERE is_lahore</code>
312 strict-label	<code>... WHERE is_minority_targeted</code>
LR ROC AUC 0.841	<code><see project reports></code>
RF Precision@20 0.35	<code><see project reports></code>

Claim in this report	Where it was computed
PS-week LR ROC AUC 0.7154	<see project reports>
Prophet Punjab MAPE 9.54%	<see project reports>
18.9% within ± 1 day of major event	EDA + <code>religious_calendar_features</code>
Top-15 forecasted PSes	<code>minority_psweek_forward</code> table
Lahore-skew (Moutra rank 126)	Audit script

End of report.